Security Architecture

Every security control that protects PhysioGuidance production — from edge DDoS through per-service IAM, argon2 password hashing, PII encryption, malware scanning, secrets management, and login lockout — all verified against production.

PREPARED FOR

PhysioGuidance Client Team

DATE

2026-08-17

DEPLOYED COMMIT

`d396b040`

PHYSIOGUIDANCE

Riyadh · Saudi Arabia
me-central2 · GCP Cloud Run

Table of contents

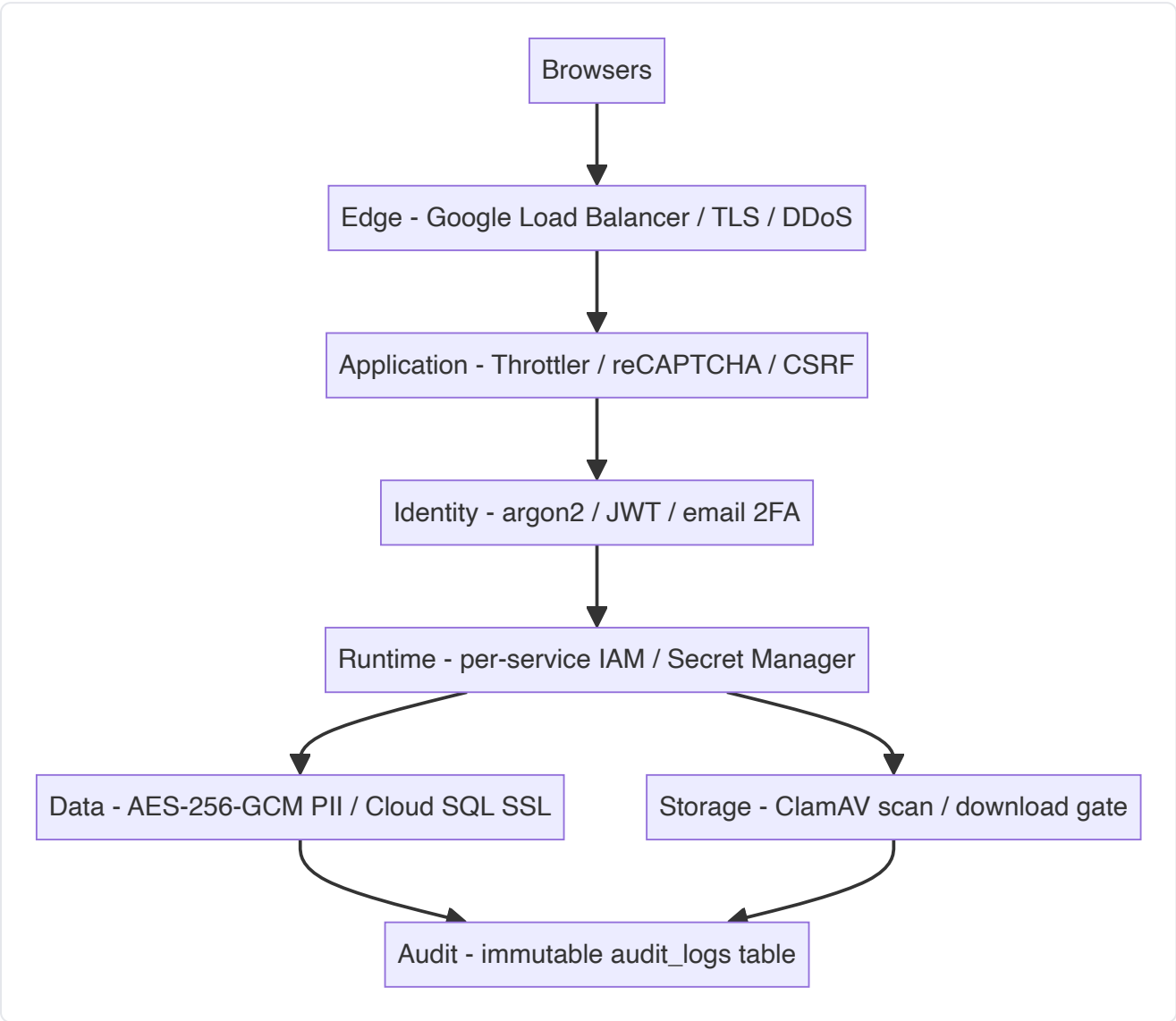
1. Executive summary
2. Layered defense — bird's-eye view
3. Edge — DDoS + TLS + HTTP security headers
4. Application layer — throttling, reCAPTCHA, CSRF
5. Authentication — argon2, JWT, refresh tokens
6. Login lockout — end-to-end enforcement
7. Two-factor authentication (email OTP + trusted devices)
8. Authorization — role-based access
9. Session management
10. PII encryption at rest (AES-256-GCM)
11. Content Security Policy
12. Secrets management (Secret Manager)
13. Per-service IAM (least-privilege service accounts)
14. Deployment identity — Workload Identity Federation
15. Database security (Cloud SQL)
16. Malware scanning + download gate
17. Network isolation
18. Error monitoring (Sentry)
19. Audit logging
20. Owner/admin IAM audit
21. Component status matrix

1. Executive summary

PhysioGuidance uses defense-in-depth: Google's global load balancer at the edge, application throttling + reCAPTCHA + CSRF at the request layer, argon2 password hashing + JWT auth + email 2FA at the identity layer, per-service least-privilege IAM at the runtime layer, AES-256-GCM PII encryption at the data layer, and an Eventarc-triggered ClamAV scanner covering all user-upload buckets with a download gate that blocks files the scanner has not cleared.

Every layer has a specific control and an evidence source in the repo. The full controls matrix is in §21 using Amna's four-category vocabulary.

2. Layered defense — bird's-eye view



Each subsequent section details one layer.

3. Edge — DDoS + TLS + HTTP security headers

- **Google Global Load Balancer** fronts Cloud Run services (web, api, cms). Automatic edge-level DDoS mitigation at Google's network layer.
- **TLS 1.2+** enforced. Managed certificates via GCP for custom domains.
- **HTTP/2** on inbound; HTTP/1.1 to Cloud Run backend.
- **No public SSH/RDP surface** anywhere on Cloud Run (no VMs to compromise).

Application-side response headers set by the NestJS bootstrap in main.ts:

Header	Value	Scope
`Strict-Transport-Security`	`max-age=31536000; includeSubDomains`	Production only
`X-Content-Type-Options`	`nosniff`	Every response
`Referrer-Policy`	`strict-origin-when-cross-origin`	Every response
`X-Frame-Options`	`DENY`	Every response except `/storage/files/*`
`Content-Security-Policy: frame-ancestors`	`'self' <CORS_ORIGIN>`	`/storage/files/*` only, so the FE can embed uploaded PDFs while JSON endpoints stay unframable

Cloud Armor / WAF was scoped to documentation-only review in Amna doc §9; edge + application defenses cover the OWASP surface for a Cloud Run app of this scale.

---

4. Application layer — throttling, reCAPTCHA, CSRF

4.1 Global rate limiting (NestJS Throttler)

Configured in app.module.ts:

```
ThrottlerModule.forRoot([
  { name: 'default', ttl: 60_000, limit: 600 },
]);
```

- 600 requests/min/IP. Wide budget for dashboard fan-out while still capping scraping / brute force.
- `trust proxy` is enabled so `req.ip` reflects the real client behind the load balancer instead of collapsing every user into one bucket.
- Sensitive routes tighten per-route via `@Throttle({ default: { ttl, limit } })` — for example webhook receivers, payment quote endpoints, reconciliation cron, escalations.

4.2 reCAPTCHA v3 on public forms

Verified server-side by recaptcha.service.ts using `RECAPTCHA_SECRET_KEY` from Secret Manager. Enforced actions: `register`, `login`, `forgot-password`, `reset-password`, `password-setup`. Missing secret refuses the action rather than silently allowing it.

4.3 CSRF double-submit cookie

Implemented in csrf.middleware.ts. A random `csrf` cookie is set on first request; state-changing methods require the same value echoed in the `X-CSRF-Token` header. Bearer-token requests and signed webhooks bypass the check — they are already CSRF-safe.

---

5. Authentication — argon2, JWT, refresh tokens

5.1 Password hashing

- **argon2** via the `argon2` npm package (memory-hard, resistant to GPU brute force).
- Salt automatically randomized per hash.
- Verification takes ~100ms on Cloud Run — deliberate latency cost for security.
- Same hasher used for signup, admin-created users, password reset, and password change.

5.2 JWT architecture

Two-token pattern. Lifetimes are admin-configurable via Settings → Authentication; the defaults below apply when the setting is unset.

Token	Purpose	Default lifetime	Admin setting	Storage
Access token	Authorize API calls	15 minutes	`sessionTimeoutMinutes`	Returned in login response, sent as `Authorization: Bearer`
Refresh token	Get new access tokens	7 days	`refreshTokenDays`	HTTP-only `refreshToken` cookie

- Access tokens are stateless (no DB lookup per request).
- Refresh tokens are persisted in `refresh_tokens` with `token`, `expiresAt`, `deviceInfo`, `ipAddress`, `isRevoked`, `revokedAt` for per-device revocation.

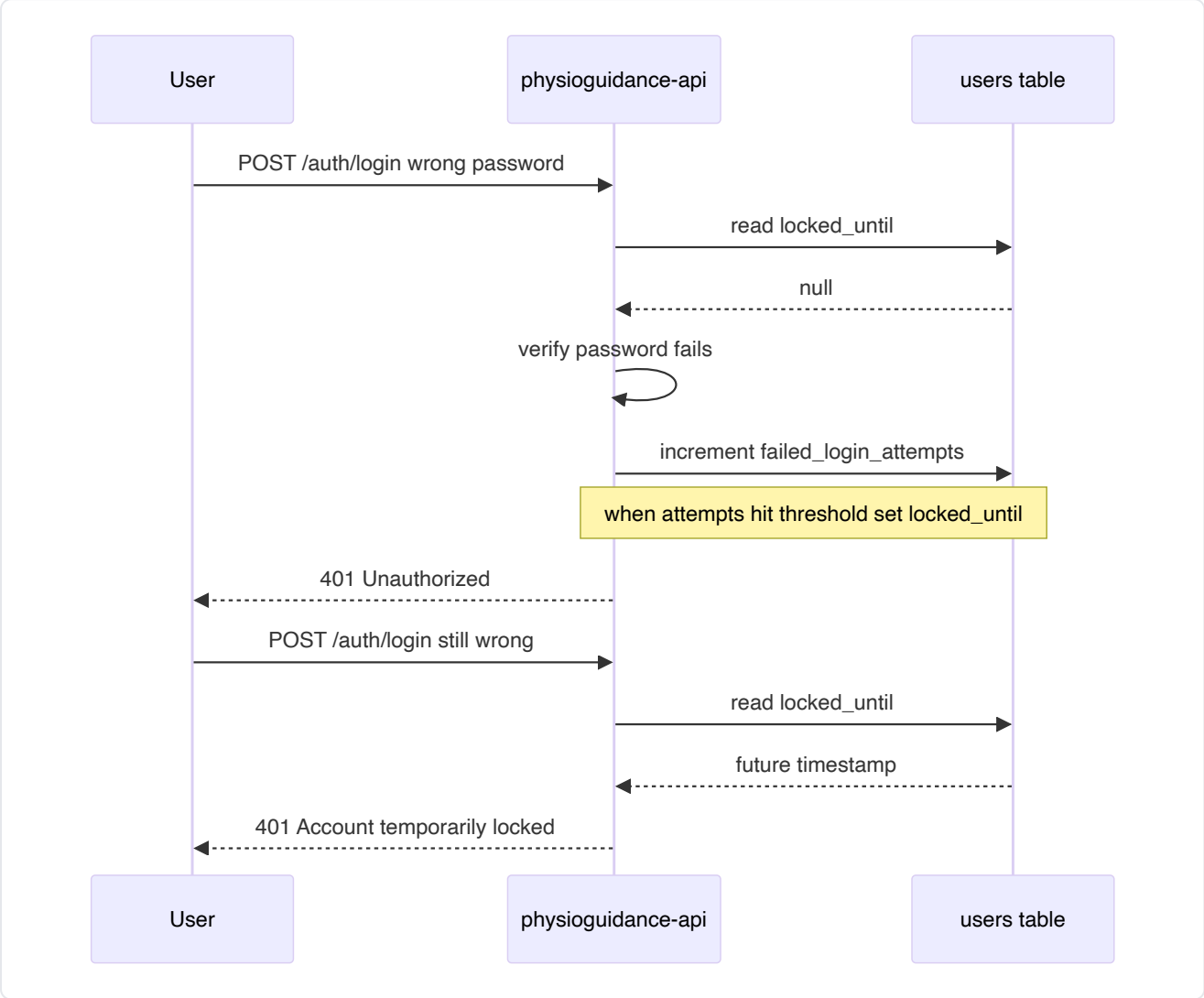
5.3 Refresh token rotation + revocation

- Every `/auth/refresh` call issues a new refresh token and revokes the previous one.
- Logout revokes the current refresh token; "Logout all devices" revokes every refresh token for the user.
- Role changes and user suspension call the internal `revokeAllUserSessions` helper, which revokes refresh tokens and clears trusted devices in one go.

---

6. Login lockout — end-to-end enforcement

Amna doc §9 specifically asked to verify lockout is wired end-to-end. Implemented and active.



Configuration (from Settings → Authentication, resolved in admin-settings.service.ts):

- Threshold: `lockoutAttempts`, default 5
- Duration: `lockoutMinutes`, default 15
- Reset: on successful login, `failed_login_attempts` is set to 0 and `locked_until` is cleared

Verified in auth.service.ts — pre-verify lockout check + post-failure increment + threshold lock.

7. Two-factor authentication (email OTP + trusted devices)

2FA is enforced by default on every account and uses **email one-time passcodes** delivered through the platform's transactional email provider. It is implemented in two-factor.service.ts.

7.1 Challenge flow

- After password success, the API generates a 6-digit code with `crypto.randomInt`.
- The code is SHA-256 hashed and stored in `two_factor_challenges` (`code_hash`, `fingerprint`, `expires_at`, `attempts`, `is_used`).
- The plain code is emailed to the account address via EmailService.sendOTP using the user's locale (en/ar).
- The user submits the code; the API SHA-256 hashes and compares.

7.2 Limits

- Code expiry: 10 minutes.
- Max attempts per challenge: 3, then the challenge is marked used.
- Cooldown between resends: 60 seconds per user; earlier unused challenges are invalidated when a new one is issued.

7.3 Trusted devices

- Fingerprint = SHA-256 of `User-Agent + device cookie`.
- On successful 2FA, the user can trust the current device — stored in `trusted_devices` with parsed device name (browser + OS) and IP.
- Trusted devices skip the OTP prompt on subsequent logins.
- Users see and revoke devices from Settings → My Security; admin can revoke a target user's devices, which also revokes the linked refresh tokens.
- Disabling 2FA revokes all trusted devices for the user.

7.4 Toggle + audit

- `POST /auth/2fa/toggle` writes to `users.two_factor_enabled` and logs `auth.2fa_enabled` / `auth.2fa_disabled` to `audit_logs`.

8. Authorization — role-based access

Roles defined on `users.active_role` and `user_roles`: `STUDENT`, `INSTRUCTOR`, `TEACHING_ASSISTANT`, `MANAGER`, `ADMIN`, `FINANCE`, `SUPPORT`, `SUPER_ADMIN`.

Enforcement:

- Every non-`@Public()` endpoint runs through `JwtAuthGuard` (see guards/).
- Role-restricted endpoints add `@Roles(...)` checked by `RolesGuard`.
- SUPER_ADMIN can promote/demote roles; regular ADMIN cannot promote to SUPER_ADMIN.
- Role changes revoke all refresh tokens and trusted devices so old sessions cannot outlive the new permission set.

---

9. Session management

- Refresh token stored in the `refreshToken` HTTP-only cookie (browser JS cannot read it).
- `secure=true` in production (HTTPS-only).
- `sameSite='none'` in production so the frontend on `physioguidance.org` can send the cookie to `api.physioguidance.org`; `'lax'` in local development.
- `maxAge = 7 days` (matches refresh token TTL).
- No server-side session store — access tokens are stateless JWTs; only refresh tokens hit the DB, and logout revokes them.

---

10. PII encryption at rest (AES-256-GCM)

Implemented in common/crypto.ts.

- Key source: `PII_ENCRYPTION_KEY` in Secret Manager (32-byte key, base64 or 64-char hex).
- Cipher: AES-256-GCM with a 12-byte random IV per encryption; the GCM auth tag guarantees tamper-detection on read.
- Storage format: `v1:<base64-iv>:<base64-authTag>:<base64-ciphertext>`. The version prefix reserves room for future rotation without a lock-step migration.
- Backward-compatible: legacy plaintext rows without the `v1:` prefix are returned as-is, so a mixed table reads cleanly during rollout.
- Fields protected via `decryptUserPII`: `phone`, `nationalId`, `scfhsNumber` on the users table.

Rotation: key rotation requires re-encrypting existing rows. Do not rotate solo — full procedure in secrets-rotation-plan.md §3.

---

11. Content Security Policy

Two-layer CSP:

- **Frontend (Next.js).** middleware.ts generates a per-request nonce, builds an app-specific CSP, and sets `content-security-policy` on both the request headers (so Next tags its own scripts with the nonce) and the response headers. Global hardening headers on every route come from next.config.ts.
- **Backend (`/storage/files/*`).** `Content-Security-Policy: frame-ancestors 'self' <CORS_ORIGIN>` so the frontend can embed uploaded PDFs and images in preview modals while every other route stays `X-Frame-Options: DENY` .

---

12. Secrets management (Secret Manager)

All runtime secrets live in GCP Secret Manager. Full inventory + per-secret rotation procedure in secrets-rotation-plan.md.

Key points:

- **No secrets in code.** No secrets in `.env` files in the repo. GitHub Actions holds only WIF pointers + project ID (non-sensitive).
- **No secrets on developer machines.** All runtime secrets are pulled from Secret Manager at Cloud Run container start via `--set-secrets` .
- **Build-time secrets** (for example `SENTRY_AUTH_TOKEN` for source-map upload) are read from Secret Manager during CI docker build.
- **Per-service accessor grants.** Each runtime service account has `roles/secretmanager.secretAccessor` bound only to the specific secrets it needs, not project-wide.
- Deploy scripts (`infra/backend/deploy.sh` , `infra/worker/deploy.sh` , `infra/frontend/deploy.sh` , `infra/strapi/deploy.sh`) describe each secret and skip cleanly if it is unset — a missing optional secret degrades a feature (for example admin Error Monitoring), never the deploy.

Representative secrets: `DATABASE_URL` , `JWT_SECRET` , `JWT_REFRESH_SECRET` , `PII_ENCRYPTION_KEY` , `RECAPTCHA_SECRET_KEY` , `NEXT_PUBLIC_RECAPTCHA_SITE_KEY` , `RESEND_API_KEY` , `MAIL_FROM` , `MAIL_REPLY_TO` , `SENTRY_DSN_BACKEND` , `NEXT_PUBLIC_SENTRY_DSN` , `SENTRY_AUTH_TOKEN` , `SENTRY_API_TOKEN` , `SENTRY_WEBHOOK_SECRET` ,

`REDIS_HOST`, `REDIS_CA_CERT`, `OPENAI_API_KEY`, `ZOOM_ACCOUNT_ID`, `RECONCILIATION_CRON_TOKEN`, `NELC_LRS_PASSWORD`, `NELC_LRS_PASSWORD_PROD`, `STRAPI_DATABASE_PASSWORD`, plus payment gateway keys (`MOYASAR_*` test + live, `TAMARA_*`, `TABBY_*`).

---

13. Per-service IAM (least-privilege service accounts)

Amna doc §9 required removing default compute SA usage. Implemented 2026-08-15.

Every Cloud Run service and every Cloud Run Job runs under a dedicated service account:

Service account	Attached to	Least-privilege scope
`web-runtime@`	web (frontend)	Secret access for build-time public keys
`api-runtime@`	api	`cloudsql.client`, `secretmanager.secretAccessor` on bound secrets, `storage.objectAdmin` on the three platform buckets, `cloudtrace.agent`, `monitoring.metricWriter`
`worker-runtime@`	worker + all Cloud Run Jobs + Scheduler	Same shape as api-runtime, plus job execution
`strapi-runtime@`	cms (Strapi)	`cloudsql.client`, own bucket storage, secret access
`malware-runtime@`	malware-scanner + Eventarc triggers	`storage.objectAdmin` on the three watched buckets + `physioguidance-quarantine`, `eventarc.eventReceiver` at project level, `run.invoker` on the scanner service

`roles/editor` on the default compute SA has been removed. Optional cleanup script for the residual Google-managed platform bindings: least-privilege-service-accounts-for-amna.sh.

---

14. Deployment identity — Workload Identity Federation

GitHub Actions authenticates to GCP through Workload Identity Federation — no long-lived JSON service-account keys anywhere.

Every deploy workflow — deploy-backend.yml, deploy-worker.yml, deploy-frontend.yml, deploy-strapi.yml — declares `id-token: write` and runs `google-github-actions/auth` with:

```
workload_identity_provider: ${{ secrets.WIF_PROVIDER }}
service_account:            ${{ secrets.WIF_SERVICE_ACCOUNT }}
```

The GitHub OIDC token is exchanged for a short-lived GCP access token that impersonates the deploy service account. Anthropic's WIF setup ties the provider to the specific GitHub org + repo, so leaked WIF pointer secrets cannot be reused from another repository.

---

15. Database security (Cloud SQL)

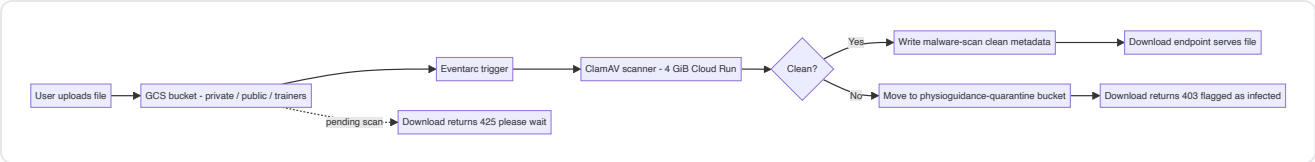
Verified via `gcloud sql instances describe physioguidance-db-me`:

Control	State
SSL required for all connections	Implemented (`requireSsl=True`)
Private IP + Cloud SQL Auth Proxy for admin access	Implemented
Runtime services connect via Unix socket at `/cloudsql/PROJECT:REGION:INSTANCE` (Cloud Run `--add-cloudsql-instances`)	Implemented
Migrations run through `cloud-sql-proxy` from `infra/scripts/migrate-production.sh` — bypasses public IP entirely	Implemented
Deletion protection enabled	Implemented
Point-in-Time Recovery (PITR)	Implemented (~5 min RPO)
Automated daily backups	Implemented (7-day retention)
Backup restore drill verified	Implemented (2026-08-17 — clone into me-central2-b, data verified, deleted)
Password rotation procedure	Documented in secrets-rotation-plan.md §3

---

16. Malware scanning + download gate

Full detail in malware-unscanned-file-gate.md. Scanner source: infra/malware-scanner/main.js.



16.1 Scanner service

- Cloud Run service `physioguidance-malware-scanner`, deployed with 4 GiB memory, 2 CPU, 540s timeout, `--no-allow-unauthenticated`, running as `malware-runtime@`.
- ClamAV signatures baked into the container image via `freshclam` at build time; re-running `infra/malware-scanner/deploy.sh` refreshes signatures.
- Eventarc triggers `malware-scan-physioguidance-{private,public,trainers}` filter `google.cloud.storage.object.v1.finalized` per bucket and impersonate `malware-runtime@` when invoking the scanner. `run.invoker` is granted on the scanner service so deliveries do not silently 401.
- Files over 50 MiB skip scanning and emit a WARNING log entry (Cloud Run `/tmp` is RAM-backed).

16.2 Verdict recording

- **Clean:** the scanner writes object custom metadata `malware-scan=clean` + `malware-scan-at=<ISO timestamp>` and logs an INFO entry.
- **Infected:** the scanner moves the object to `gs://physioguidance-quarantine/<original-bucket>/<name>` and emits an ERROR entry with the detected threat name. The `Malware detected` alert policy (see alert-malware-detected.json) routes it to security email channels.

16.3 Download gate

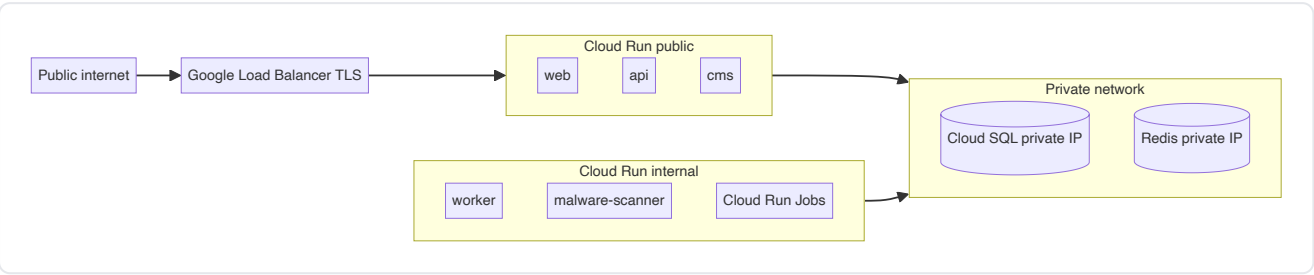
Implemented in StorageService.getScanStatus and enforced by every signed-URL and streaming endpoint that passes `requireCleanScan: true`:

- **Clean** (`malware-scan=clean` metadata) → download proceeds
- **Pending** (metadata not yet written) → HTTP 425 Too Early with a "please wait" message
- **Quarantined** (object 404 in original bucket because the scanner moved it) → HTTP 403 Forbidden

Bank-transfer receipt download and the admin review modal pass `requireCleanScan: true`; the public bucket is exempt (system-generated invoices only).

---

17. Network isolation



- Only web, api, and cms accept public traffic (`--allow-unauthenticated`).
- Worker, malware-scanner, and Cloud Run Jobs are deployed `--no-allow-unauthenticated` ; they are invoked via Cloud Tasks, Cloud Scheduler, or Eventarc using IAM-authenticated calls.
- Cloud SQL + Redis are reached from Cloud Run through the Serverless VPC Connector `physio-vpc-connector-me` ; the DB connection itself goes through the Cloud SQL socket at `/cloudsql/CONNECTION_NAME` .
- No firewall rules exposing SSH/RDP — Cloud Run has no VMs.

---

18. Error monitoring (Sentry)

- Backend uses `@sentry/nestjs` initialized in instrument.ts with `nodeProfilingIntegration` ; DSN from `SENTRY_DSN_BACKEND` . The global `AllExceptionsFilter` reports unhandled exceptions.
- Frontend uses `NEXT_PUBLIC_SENTRY_DSN` ; source maps are uploaded during CI using `SENTRY_AUTH_TOKEN` .
- Sentry alert rules POST to `/api/webhooks/sentry/alert` ; the webhook signature-verifies with `SENTRY_WEBHOOK_SECRET` , then fans out to in-app notifications through error-monitoring-notifications.service.ts. Admins browse issues in the built-in Error Monitoring dashboard, which reads the Sentry API with `SENTRY_API_TOKEN` .
- Combined with Cloud Logging alert policies (see infra/monitoring/) — `alert-5xx-spike` , `alert-failed-login-burst` , `alert-suspicious-403` , `alert-malware-detected` — this pipeline supports breach detection within the PDPL Article 27 72-hour notification window.

---

19. Audit logging

Every meaningful action writes to `audit_logs` via AuditLogService. Per-request IP + user-agent are captured through `RequestContextService` in an AsyncLocalStorage store so every log entry carries them without call-site plumbing.

Category	Examples
Authentication	login success, login failure, logout, 2FA enable, password change
PDPL	consent grant/withdraw, right-to-erasure request, data export
Admin	role change, user suspend, backup create, template edit, feature flag toggle
Finance	payment approve, refund initiate, invoice regenerate
Content	course publish, course archive, material upload

Admins query at `/dashboard/admin/logs` — filter by user, action type, date range. CSV / XLSX / PDF export for auditors. Retention: indefinite (compliance evidence).

---

20. Owner/admin IAM audit

Amna doc §9 asked to confirm current IAM ownership.

Account	Roles	Notes
`amna@physioguidance.org`	Project Owner	Client owner
`ghulam@physioguidance.org`	Project Owner (handover holdout)	To be removed post-handover sign-off. Script ready.
`github-actions-deployer@` (WIF-impersonated)	Least-privilege deploy scope	Used only by GitHub Actions via Workload Identity Federation
5 runtime SAs	See §13	Per-service scope
Default compute SA	Reduced (editor role removed)	Residual inherited platform bindings remain (optional cleanup)

2FA on privileged accounts: verified enabled on ghulam@ + amna@ Google accounts.

---

21. Component status matrix

Using Amna's four-category vocabulary. Only items Amna asked about + items shipped as part of this handover.

Control	Status	Notes
Cloud SQL SSL-only	Implemented	`requireSsl=True` verified 2026-08-17
Cloud SQL deletion protection	Implemented	Verified 2026-08-17
Cloud SQL PITR	Implemented	Verified 2026-08-17
Runtime DB access via `/cloudsql/` socket	Implemented	`--add-cloudsql-instances` on every service
Per-service SAs (web, api, worker, cms, scanner)	Implemented	Default compute SA no longer used for runtime
`roles/editor` removed from default compute SA	Implemented	Residual Google-managed bindings remain (optional post-handover cleanup)
Workload Identity Federation for GitHub Actions	Implemented	No long-lived JSON keys anywhere
2FA on privileged accounts	Implemented	Verified on Ghulam + Amna Google accounts
App-level 2FA (email OTP + trusted devices)	Implemented	6-digit OTP, 10 min expiry, 3 attempts, 60s resend cooldown
argon2 password hashing	Implemented	Used on signup, admin-create, reset, change
JWT access token 15 min + refresh 7 days, rotated on refresh	Implemented	Both admin-configurable
Refresh token revocation on logout / role change / suspend	Implemented	`revokeAllUserSessions` helper
Login lockout wired end-to-end	Implemented	Threshold + duration configurable; verified in auth.service.ts
reCAPTCHA v3 on public forms	Implemented	Missing secret refuses the action
CSRF double-submit cookie	Implemented	Bearer + signed webhook requests bypass
Rate limiting (NestJS Throttler, 600/min/IP + per-route tighteners)	Implemented	`trust proxy` on so per-user buckets survive Cloud Run
PII encryption at rest (AES-256-GCM)	Implemented	`phone`, `nationalId`, `scfhsNumber` on users
HSTS + X-Content-Type-Options + Referrer-Policy + X-Frame-Options	Implemented	Set globally in `main.ts`
Frontend nonce-based CSP	Implemented	Per-request nonce in `middleware.ts`
Backend CSP `frame-ancestors` on `/storage/files/*`	Implemented	Enables preview embed without weakening JSON routes
Malware scan on user uploads	Implemented	Eventarc-triggered ClamAV, 4 GiB memory

Control	Status	Notes
Malware download gate on sensitive paths	Implemented	403 quarantined, 425 pending, clean allowed
File size limit enforced pre-upload	Implemented	10 MB support attachments, 5 MB avatar, size-limited receipt
No SSH/RDP surface	Implemented	Cloud Run has no VMs
Sentry error tracking (backend + frontend)	Implemented	Webhook signature-verified, admin dashboard live
Breach detection pipeline (Sentry + Cloud Logging alerts)	Implemented	Supports PDPL Article 27 72-hour window
Cloud Armor / WAF	Not Applicable	Amna doc §9 scoped this documentation-only

22. Where to look next

- **How PDPL rights are exposed to users** → pdpl-compliance.md
- **How secrets are rotated** → secrets-rotation-plan.md
- **How the malware gate handles pending/infected files** → malware-unscanned-file-gate.md
- **How monitoring surfaces security events** → monitoring-alerting.md
- **How backup + restore work** → regional-failure-playbook.md